

POLITECHNIKA ŚWIĘTOKRZYSKA Wydział Elektrotechniki, Automatyki i Informatyki
--

Wprowadzenie do cyberbezpieczeństwa
--

Wykonali: Filip Turno Dmytro Rudnytskyi Radek Kopacz	Temat: System logowania
---	------------------------------------

Spis treści

1.	WSTĘP	3
2.	IMPLEMENTACJA MECHANIZMÓW BEZPIECZEŃSTWA	4
2.1.	BEZPIECZNE PRZECHOWYWANIE HASEŁ (ARGON2ID)	4
2.2.	UWIERZYTELNIANIE DWUSKŁADNIKOWE (TOTP)	4
2.3.	WSKAŹNIK SIŁY HASŁA I ANALIZA ENTROPII	5
2.4.	OCHRONA PRZED BRUTE FORCE (ACCOUNT LOCKOUT)	6
3.	ANALIZA ZAGROŻEŃ	8
4.	INSTRUKCJA URUCHOMIENIA SYSTEMU W ŚRODOWISKU INTELLIJ IDEA	9
5.	PODSUMOWANIE	10

1. WSTĘP

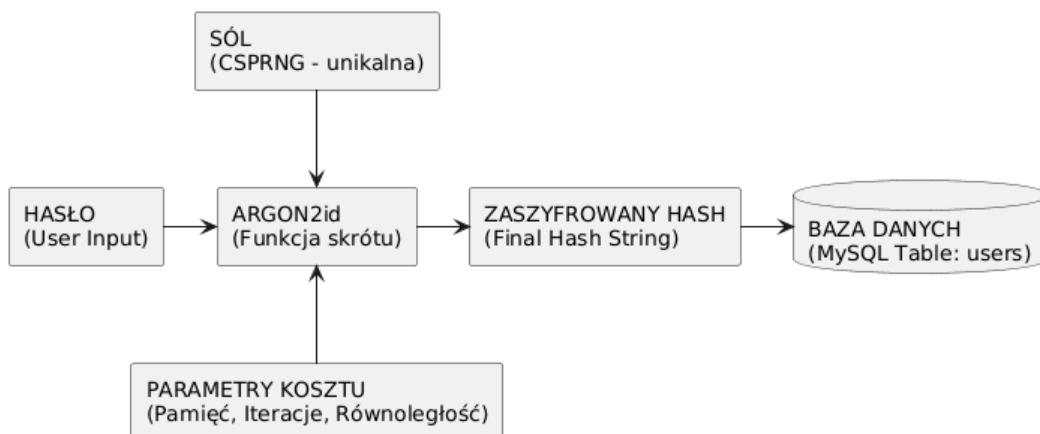
Celem projektu było stworzenie bezpiecznego systemu uwierzytelniania i autoryzacji opartego na standardach OWASP ASVS 4.0. System został zbudowany w architekturze klient-serwer. Warstwa wizualna (Frontend) została stworzona w bibliotece React z wykorzystaniem narzędzia Vite oraz Tailwind CSS dla zapewnienia responsywnego interfejsu. Warstwa logiczna (Backend) opiera się na środowisku Node.js z frameworkiem Express, natomiast dane przechowywane są w relacyjnej bazie danych MySQL.

System implementuje zaawansowane mechanizmy ochrony, takie jak nowoczesne hashowanie haseł, uwierzytelnianie wieloskładnikowe (MFA) oraz ochronę przed automatycznymi atakami typu Brute Force.

Tabela 1.1. Tabela z adresacją i portami

Element	Narzędzie	Adres / Port	Funkcja
Frontend	React + Vite	http://localhost:5173	Interfejs użytkownika
Backend	Node.js + Express	http://localhost:3000	Logika biznesowa i API
Baza danych	MySQL	localhost:3306	Przechowywanie danych

2. IMPLEMENTACJA MECHANIZMÓW BEZPIECZEŃSTWA



Rysunek 2.1 Schemat procesu generowania bezpiecznego skrótu hasła z wykorzystaniem algorytmu Argon2id. W odróżnieniu od klasycznych funkcji skrótu, system uwzględnia nie tylko sól (zapobiegającą atakom Rainbow Tables), ale również konfigurowalne parametry kosztu pamięciowego i obliczeniowego, co skutecznie mityguje ataki przy użyciu układów GPU/ASIC.

W niniejszym rozdziale przedstawiono kluczowe funkcjonalności systemu, zidentyfikowane problemy w zakresie cyberbezpieczeństwa oraz zastosowane rozwiązania techniczne wraz z fragmentami kodu.

2.1. BEZPIECZNE PRZECHOWYWANIE HASEŁ (ARGON2ID)

Problem: Przechowywanie haseł w formie jawnej (plain-text) lub przy użyciu słabych funkcji skrótu (np. MD5, SHA1) naraża użytkowników na natychmiastowe przejęcie konta w przypadku wycieku bazy danych.

Rozwiązanie: Zastosowano algorytm Argon2id, który jest obecnie rekomendowany przez OWASP oraz zwycięzcę konkursu Password Hashing Competition. Algorytm ten posiada wbudowany mechanizm solenia haseł i jest odporny na ataki typu side-channel oraz optymalizację przy użyciu układów GPU.

```
const hashedPassword = await argon2.hash(password);
await pool.execute('INSERT INTO users (username, email, password_hash) VALUES
(?, ?, ?)',
[username, email, hashedPassword]);
```

	id	username	email	password_hash	role
1	14	Viktor Przyg...	viktorprzygodzki20...	\$argon2id\$v=19\$m=65536,t=3,p=4\$g+lVL...	user
2	15	Dmytro Rudny...	rdima7934@gmail.com	\$argon2id\$v=19\$m=65536,t=3,p=4\$FaQJJ...	user

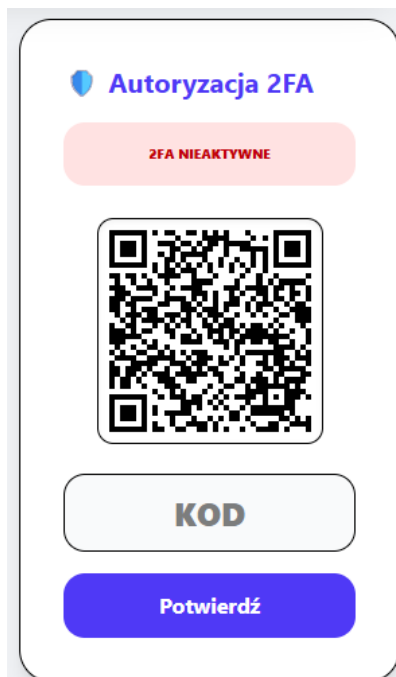
Rysunek 2.1. Widok zahashowanych danych użytkowników w bazie MySQL.

2.2 UWIERZYTELNIANIE DWUSKŁADNIKOWE (TOTP)

Problem: Tradycyjne logowanie oparte wyłącznie na hasle jest podatne na ataki typu phishing lub przechwycenie danych logowania.

Rozwiązanie: Zaimplementowano standard TOTP (Time-based One-Time Password). Użytkownik może powiązać swoje konto z aplikacją mobilną (np. Google Authenticator) poprzez skanowanie kodu QR. Serwer weryfikuje 6-cyfrowy kod generowany na podstawie wspólnego sekretu i aktualnego czasu.

```
const verified = speakeasy.totp.verify({
  secret: user.two_factor_secret,
  encoding: 'base32',
  token: totpToken,
  window: 1
});
```



Rysunek 2.2. Interfejs aktywacji uwierzytelniania dwuskładnikowego.

2.3 WSKAŹNIK SIŁY HASŁA I ANALIZA ENTROPII

Problem: Tradycyjne reguły złożoności (wymóg cyfr czy znaków specjalnych) często prowadzą do tworzenia haseł łatwych do przewidzenia dla algorytmów (np. Hasło123!).

Rozwiązanie: Zaimplementowano dynamiczny wskaźnik siły hasła bazujący na entropii bitowej, obliczanej według wzoru: $E = L \cdot \log_2(R)$ gdzie L to długość hasła, a R to wielkość zbioru użytych znaków.

W systemie przyjęto następujące progi bezpieczeństwa:

- **< 50 bitów (Kolor czerwony):** Hasło krytycznie słabe, podatne na szybkie złamanie metodą siłową.
- **50 - 80 bitów (Kolor żółty):** Hasło o średniej sile, spełniające podstawowe wymagania, ale niezalecane dla kont z uprawnieniami wrażliwymi.
- **80 bitów (Kolor zielony):** Hasło silne, odporne na współczesne ataki słownikowe i rozproszone próby łamania metodą brute-force.

Mechanizm ten, zgodnie z ASVS 4.0, kładzie nacisk na całkowitą długość hasła (entropię) zamiast wymuszania nienaturalnych dla człowieka kombinacji znaków.

```
const verified = speakeasy.totp.verify({
  secret: user.two_factor_secret,
  encoding: 'base32',
  token: totpToken,
  window: 1
});
```

Rejestracja Użytkownika

Robert Lasek

Email

R0B3rT_-!@s3K

Entropia: 92 bits

Utwórz konto

Masz już konto? [Zaloguj się](#)

Rejestracja Użytkownika

Robert Lasek

Email

11111111111111111111111111111111

Entropia: 75 bits

Utwórz konto

Masz już konto? [Zaloguj się](#)

Rejestracja Użytkownika

Robert Lasek

Email

aaaaaaaaabcfgaweca

Entropia: 49 bits

Utwórz konto

Masz już konto? [Zaloguj się](#)

Rejestracja Użytkownika

Robert Lasek

Email

KONKek--1200

Entropia: 79 bits

Utwórz konto

Masz już konto? [Zaloguj się](#)

Rysunek 2.3. Wizualizacja siły hasła na podstawie obliczonej entropii.

2.4 OCHRONA PRZED BRUTE FORCE (ACCOUNT LOCKOUT)

Problem: Automatyczne boty mogą wykonywać setki prób logowania na sekundę, próbując odgadnąć hasło (brute-force) lub dopasować je ze znanych wycieków (credential stuffing).

Rozwiązanie: Zaimplementowano mechanizm progresywnej blokady konta. Po wykryciu 5 nieudanych prób logowania w krótkim odstępie czasu, system ustawia znacznik `lockout_until` w bazie danych.

Kluczowe parametry mechanizmu:

- **Limit prób:** 5 błędnych logowań.

- **Czas blokady:** 15 minut.
- **Cel mitygacji:** Czasowa blokada została dobrana tak, aby drastycznie zmniejszyć efektywność botów (uniemożliwiając miliony prób), jednocześnie minimalizując ryzyko ataku typu Denial of Service (DoS) na konto użytkownika. Zbyt długa lub permanentna blokada mogłaby pozwolić napastnikowi na złośliwe wykluczenie prawowitego użytkownika z systemu poprzez celowe wpisywanie błędnych haseł.

```
const valid = await argon2.verify(user.password_hash, password);
if (!valid) {
  const newAttempts = (user.failed_attempts || 0) + 1;
  if (newAttempts >= 5) {
    const lockout = new Date(Date.now() + 15 * 60000);
    await pool.execute('UPDATE users SET failed_attempts = ?,
lockout_until = ? WHERE id = ?', [newAttempts, lockout, user.id]);
    return res.status(403).json({ error: 'Blokada 15 min' });
  }
  await pool.execute('UPDATE users SET failed_attempts = ? WHERE id =
?', [newAttempts, user.id]);
  return res.status(401).json({ error: 'Błędne dane' });
}
}
```

Logowanie do Systemu

Dmytro Rudnytskyi

Daymer2706---| 🔍

Zaloguj się

Konto zablokowane. Spróbuj ponownie za 15 min.

Nie masz konta? [Zarejestruj się](#)

Rysunek 2.4. Komunikat systemowy po wykryciu próby ataku brute-force.

3. ROZDZIAŁ: ANALIZA ZAGROŻEŃ

Poniższa tabela przedstawia kluczowe zagrożenia bezpieczeństwa zidentyfikowane w systemie oraz sposób, w jaki zaimplementowana infrastruktura im zapobiega.

Tabela 3.1. Opis zagrożeń i metody ich mitygacji.

Zagrożenie	Opis i wpływ	Metoda mitygacji w aplikacji
Brute Force	Próba odgadnięcia hasła metodą prób i błędów.	Mechanizm blokady konta (lockout) oraz wymóg 2FA.
Rainbow Tables	Użycie gotowych baz skrótów do łamania haseł.	Unikalna sól dla każdego hasła w algorytmie Argon2id.
Phishing	Przejęcie hasła przez fałszywą stronę.	2FA sprawia, że samo hasło jest bezużyteczne dla atakującego.
SQL Injection	Próba wstrzyknięcia kodu do bazy danych.	Użycie zapytań parametryzowanych (Prepared Statements).
Popular Passwords	Stosowanie haseł typu "123456789012".	Weryfikacja haseł z lokalną czarną listą (common_passwords.json).

4. ROZDZIAŁ: INSTRUKCJA URUCHOMIENIA SYSTEMU W ŚRODOWISKU INTELLIJ IDEA

Aby system działał poprawnie, musisz uruchomić dwa niezależne procesy: serwer API (Backend) oraz serwer deweloperski aplikacji (Frontend).

Krok 1: Przygotowanie bazy danych

Upewnij się, że Twój serwer MySQL (np. XAMPP lub Docker) jest włączony. System automatycznie zainicjalizuje tabele przy pierwszym uruchomieniu backendu.

Krok 2: Uruchomienie serwera API (Backend)

- W IntelliJ IDEA otwórz dolną zakładkę Terminal (skrót Alt + F12).
- Upewnij się, że jesteś w głównym folderze projektu (tam, gdzie znajduje się plik server.js).
- Wpisz komendę i zatwierdź:

```
node server.js
```

Krok 3: Uruchomienie aplikacji deweloperskiej (Frontend)

1. Otwórz drugą kartę Terminala w IntelliJ (kliknij ikonę + po lewej stronie panelu Terminal).
2. Przejdź do folderu frontendowego, używając ścieżki, którą podałeś:

```
cd "<YOUR_PATH>\WprowadzenieDoCyb\frontend"
```

3. Uruchom serwer deweloperski Vite:

```
npm run dev
```

4. W konsoli pojawi się link (zazwyczaj <http://localhost:5173>). Kliknij go z przyciśniętym klawiszem Ctrl, aby otworzyć aplikację w przeglądarce.

Krok 4: Weryfikacja połączenia

- Aplikacja powinna załadować ekran logowania.
- Jeśli serwer API nie działa, w konsoli przeglądarki (F12) zobaczysz błędy połączenia (Connection Refused).
- Oba terminale w IntelliJ muszą pozostać otwarte przez cały czas pracy z programem.

5. PODSUMOWANIE

Stworzona aplikacja stanowi kompletny model bezpiecznego systemu zarządzania tożsamością. Dzięki zastosowaniu standardów OWASP ASVS 4.0 oraz nowoczesnych narzędzi, takich jak Argon2id i TOTP, system jest odporny na większość typowych ataków wymierzonych w proces uwierzytelniania. Projekt wykazał, że bezpieczeństwo aplikacji zależy nie tylko od złożoności hasła, ale przede wszystkim od bezpiecznych algorytmów przechowywania danych oraz implementacji wieloskładnikowej autoryzacji.